IMSE 785

Big Data Analytics

Pyspark

Page Rank

Shing Chang

IMSE, K-State

The codes: pyspark_pagerank.py [1/6]

- https://github.com/apache/spark/blob/master/examples/src/main/python/pagerank.py

```
18  """
19  This is an example implementation of PageRank. For more conventional use,
20  Please refer to PageRank implementation provided by graphx
21
22  Example Usage:
23  bin/spark-submit examples/src/main/python/pagerank.py data/mllib/pagerank_data.txt 10
24  """
25  from __future__ import print_function
26
27  import re
28  import sys
29  from operator import add
30
31  from pyspark.sql import SparkSession
32
```

The codes: pyspark_pagerank.py [2/6]

```
33
34  def computeContribs(urls, rank):
35      """Calculates URL contributions to the rank of other URLs."""
36      num_urls = len(urls)
37      for url in urls:
38          yield (url, rank / num_urls)
39
40
41  def parseNeighbors(urls):
42      """Parses a urls pair string into urls pair."""
43      parts = re.split(r'\s+', urls)
44      return parts[0], parts[1]
45
46
```

The codes: pyspark_pagerank.py [3/6]

```
46
47  if __name__ == "__main__":
48      if len(sys.argv) != 3:
49          print("Usage: pagerank <file> <iterations>", file=sys.stderr)
50          sys.exit(-1)
51
52      print("WARN: This is a naive implementation of PageRank and is given as an example!\n" +
53            "Please refer to PageRank implementation provided by graphx",
54            file=sys.stderr)
55
56      # Initialize the spark context.
57      spark = SparkSession\
58          .builder\
59          .appName("PythonPageRank")\
60          .getOrCreate()
61
```

The codes: pyspark_pagerank.py (flush with spark in line 57) [4/6]

```
62      # Loads in input file. It should be in format of:
63      #     URL         neighbor URL
64      #     URL         neighbor URL
65      #     URL         neighbor URL
66      #     ...
67      lines = spark.read.text(sys.argv[1]).rdd.map(lambda r: r[0])
68
69      # Loads all URLs from input file and initialize their neighbors.
70      links = lines.map(lambda urls: parseNeighbors(urls)).distinct().groupByKey().cache()
71
72      # Loads all URLs with other URL(s) link to from input file and initialize ranks of them to one.
73      ranks = links.map(lambda url_neighbors: (url_neighbors[0], 1.0))
74
```

The codes: pyspark_pagerank.py [5/6]

```
75      # Calculates and updates URL ranks continuously using PageRank algorithm.
76      for iteration in range(int(sys.argv[2])):
77          # Calculates URL contributions to the rank of other URLs.
78          contribs = links.join(ranks).flatMap(
79              lambda url_urls_rank: computeContribs(url_urls_rank[1][0], url_urls_rank[1][1]))
80
81          # Re-calculates URL ranks based on neighbor contributions.
82          ranks = contribs.reduceByKey(add).mapValues(lambda rank: rank * 0.85 + 0.15)
83
```

The codes: pyspark_pagerank.py [6/6]

```
84      # Collects all URL ranks and dump them to console.
85      for (link, rank) in ranks.collect():
86          print("%s has rank: %s." % (link, rank))
87
88      spark.stop()
```

re and sys

- re.split() – watch the following video for how split works (3:58); re stands for regular expression; there are many re expressions such as re.sub
  - https://youtu.be/KaLPVbnnDiw
- sys.argv – watch the following video for how sys argv works (4:42)
  - https://youtu.be/jhki0o54_xY

Your Turn 1/2:

- Lines 57-60: can the app “PythonPageRank” be used by others when its sparksession be downloaded? Where is this app stored?
- Line 67: what does this line do? Is there another way to read in the data?
- Line 70: examine what the following functions do: distinct().groupByKey().cache()
- Line 73: why is the initial rank value be 1 rather than 1/len(urls)?
- Line 79 what do these arguments mean url_urls_rank[1][0], url_urls_rank[1][1])?

Your Turn 2/2:

- Line 76: for iteration in range(int(sys.argv[2])): Is there another way to replace this for loop statement and achieve the same or similar effect?
- Line 82: what does this line do? Why is this necessary for the page rank algorithm?